

Skill 5

Tokenization and Parser Design

Aim

To split user input into tokens, identify delimiters and whitespace, create token structures, validate token streams, design parser logic, generate parse structures and detect syntax errors.

Requirements

- Ubuntu Linux
- GCC
- C programming
- Terminal

Step-by-Step Implementation

Step 1: Read the Input

Example:

```
ls -l /home/user
```

Step 2: Identify Delimiters

Common delimiters include:

Space

Tab

Pipe |

Redirection >

Input redirection <

Step 3: Split Input into Tokens

The command:

```
ls -l /home/user
```

is converted into:

Token 1 → ls

Token 2 → -l

Token 3 → /home/user

Step 4: Create Token Structure

A token structure can store:

Token Type

Token Value

Position

Step 5: Handle Whitespace

Multiple spaces and tabs should be treated appropriately while parsing the command.

Step 6: Validate Token Stream

The parser checks whether the generated sequence of tokens follows the expected structure.

Step 7: Design Parser Logic

Input

↓

Lexical Analysis

↓

Tokenization

↓

Syntax Validation

↓

Parse Structure

↓

Execution Structure

Step 8: Detect Syntax Errors

Examples:

ls |

```
> file  
| ls
```

are identified as invalid command structures.

Step 9: Handle Empty Commands

If the user enters only spaces or presses Enter without a command, the parser should ignore the input instead of attempting execution.

Observation

The command line was divided into meaningful tokens based on whitespace and special delimiters. Token structures were generated and validated. Invalid command arrangements and empty commands were detected before execution.

End Result

A basic lexical tokenizer and parser structure was successfully designed to process shell commands and detect syntax errors.